Peering into the kernel with retsnoop

Andrii Nakryiko

https://github.com/anakryiko/retsnoop

Meta

Why retsnoop exists?

```
$ rg '\-EINVAL' kernel/bpf | wc -l
1008
```

Why retsnoop exists?

```
$ rg '\-EINVAL' kernel/bpf | wc -l
1008
```

That’s a lot of -EINVALs.

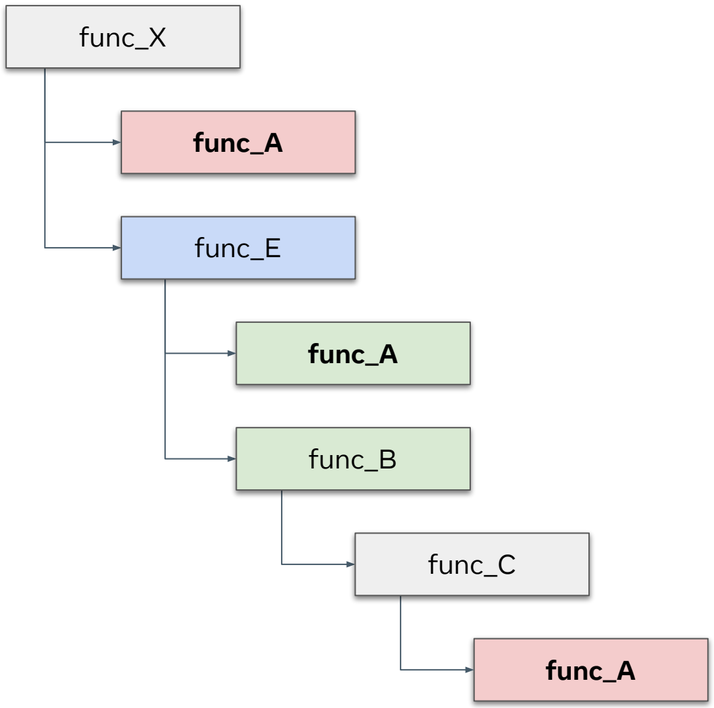
Why retsnoop exists?

Retsnoop was born to help find **which “-EINVAL”**.

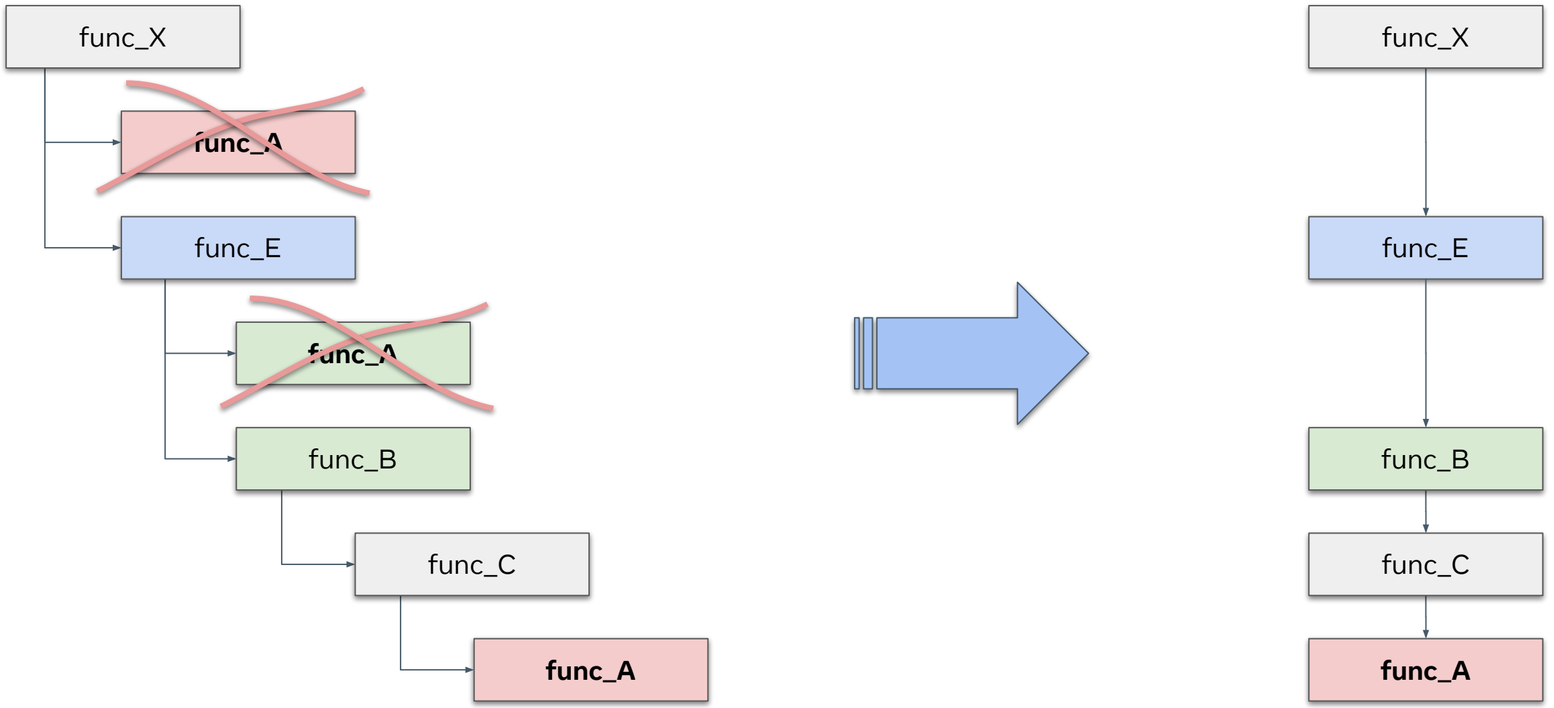
What is retsnoop?

- **Mass-tracer of kernel functions**
  - and lots of them, if necessary!
  - BPF under the hood
- **High signal-to-noise ratio**
  - Capture errors only (by default)
  - Customizable filtering conditions
- **Capture extra context**
  - Return values
  - Functions arguments
  - LBR (Last Branch Records) data

Session (limiting the scope)

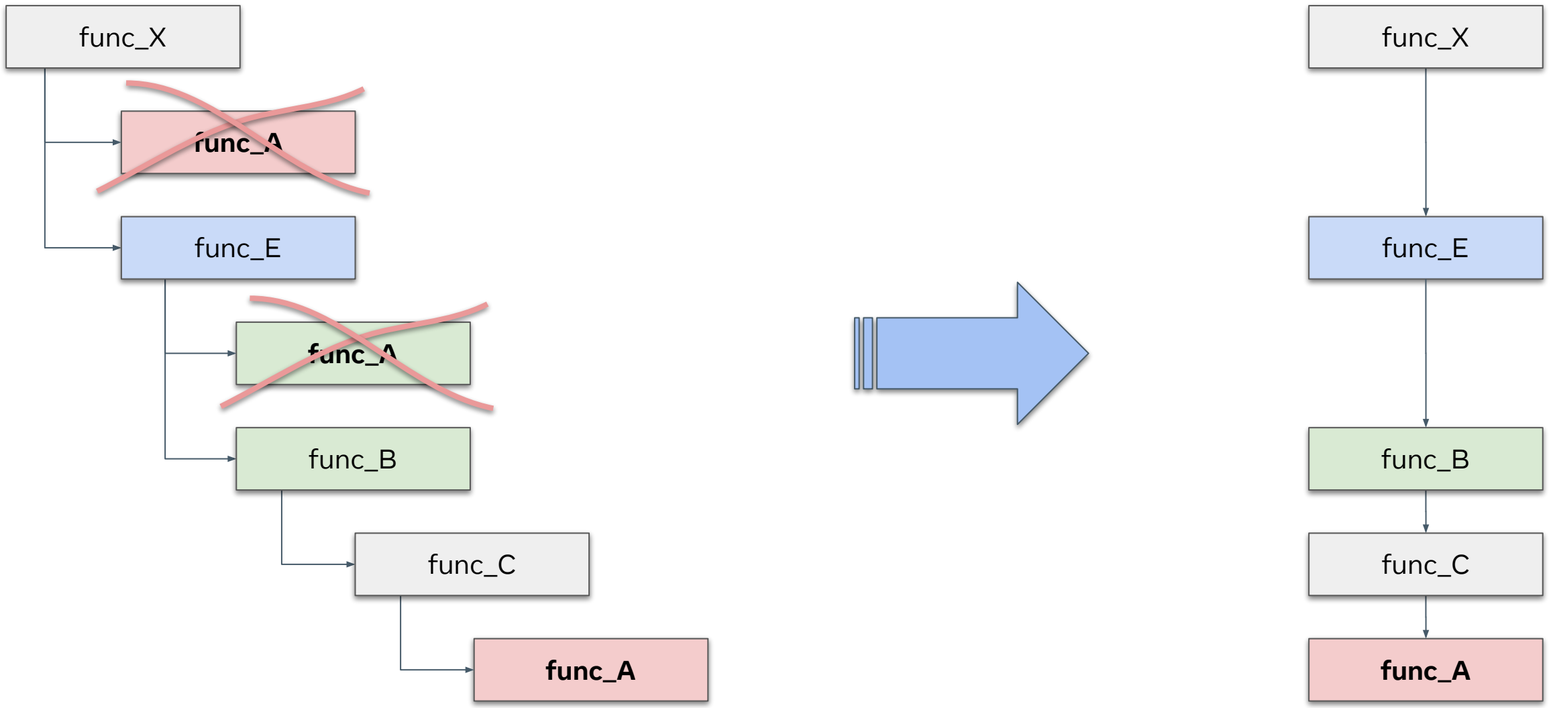


Session (limiting the scope)



Entry vs non-entry functions

```
sudo retsnoop -e func_E -a func_A -a func_B
```



Call stack mode

```
$ sudo retsnoop -e '*sys_bpf' -a ':kernel/bpf/*.c'
Receiving data...
06:55:21.096414 -> 06:55:21.096690 TID/PID 2125519/2125519 (simfail/simfail):

                    entry_SYSCALL_64_after_hwframe+0x46  (entry_SYSCALL_64 @ arch/x86/entry/entry_64.S:120)
                    do_syscall_64+0x3d                   (arch/x86/entry/common.c:80)
                    . do_syscall_x64                     (arch/x86/entry/common.c:50)
   273us [-ENOMEM]  __x64_sys_bpf+0x18                   (kernel/bpf/syscall.c:5672)
                    . __se_sys_bpf                       (kernel/bpf/syscall.c:5672)
                    . __do_sys_bpf                       (kernel/bpf/syscall.c:5674)
   263us [-ENOMEM]  __sys_bpf+0x1cef                     (kernel/bpf/syscall.c:5549)
                    . map_create                         (kernel/bpf/syscall.c:1268)
   220us [-ENOMEM]  array_map_alloc+0x153                (kernel/bpf/arraymap.c:149)
                    . bpf_array_alloc_percpu             (kernel/bpf/arraymap.c:40)
     2us [NULL]     bpf_map_alloc_percpu
```

Call stack mode (annotated)

```
$ sudo retsnoop -e '*sys_bpf' -a ':kernel/bpf/*.c'
Receiving data...
06:55:21.096414 -> 06:55:21.096690 TID/PID 2125519/2125519 (simfail/simfail):

                   entry_SYSCALL_64_after_hwframe+0x46 (entry_SYSCALL_64 @ arch/x86/entry/entry_64.S:120)
                   do_syscall_64+0x3d                  (arch/x86/entry/common.c:80)
                   . do_syscall_x64                    (arch/x86/entry/common.c:50)
   273us [-ENOMEM] __x64_sys_bpf+0x18                  (kernel/bpf/syscall.c:5672)
                   . __se_sys_bpf                      (kernel/bpf/syscall.c:5672)
                   . __do_sys_bpf                      (kernel/bpf/syscall.c:5674)
   263us [-ENOMEM] __sys_bpf+0x1cef                    (kernel/bpf/syscall.c:5549)
                   . map_create                        (kernel/bpf/syscall.c:1268)
   220us [-ENOMEM] array_map_alloc+0x153               (kernel/bpf/arraymap.c:149)
                   . bpf_array_alloc_percpu            (kernel/bpf/arraymap.c:40)
     2us [NULL]    bpf_map_alloc_percpu
```

Task info

Source code info

Stack trace

Call stack mode (annotated)

```
$ sudo retsnoop -e '*sys_bpf' -a ':kernel/bpf/*.c'
Receiving data...
06:55:21.096414 -> 06:55:21.096690 TID/PID 2125519/2125519 (simfail/simfail):

                                   entry_SYSCALL_64_after_hwframe+0x46 (entry_SYSCALL_64 @ arch/x86/entry/entry_64.S:120)
                                   do_syscall_64+0x3d                  (arch/x86/entry/common.c:80)
                                   . do_syscall_x64                    (arch/x86/entry/common.c:50)
  273us [-ENOMEM]                  __x64_sys_bpf+0x18                  (kernel/bpf/syscall.c:5672)
                                   . __se_sys_bpf                      (kernel/bpf/syscall.c:5672)
                                   . __do_sys_bpf                      (kernel/bpf/syscall.c:5674)
  263us [-ENOMEM]                  __sys_bpf+0x1cef                    (kernel/bpf/syscall.c:5549)
                                   . map_create                        (kernel/bpf/syscall.c:1268)
  220us [-ENOMEM]                  array_map_alloc+0x153               (kernel/bpf/arraymap.c:149)
                                   . bpf_array_alloc_percpu            (kernel/bpf/arraymap.c:40)
    2us [NULL]                     bpf_map_alloc_percpu
```

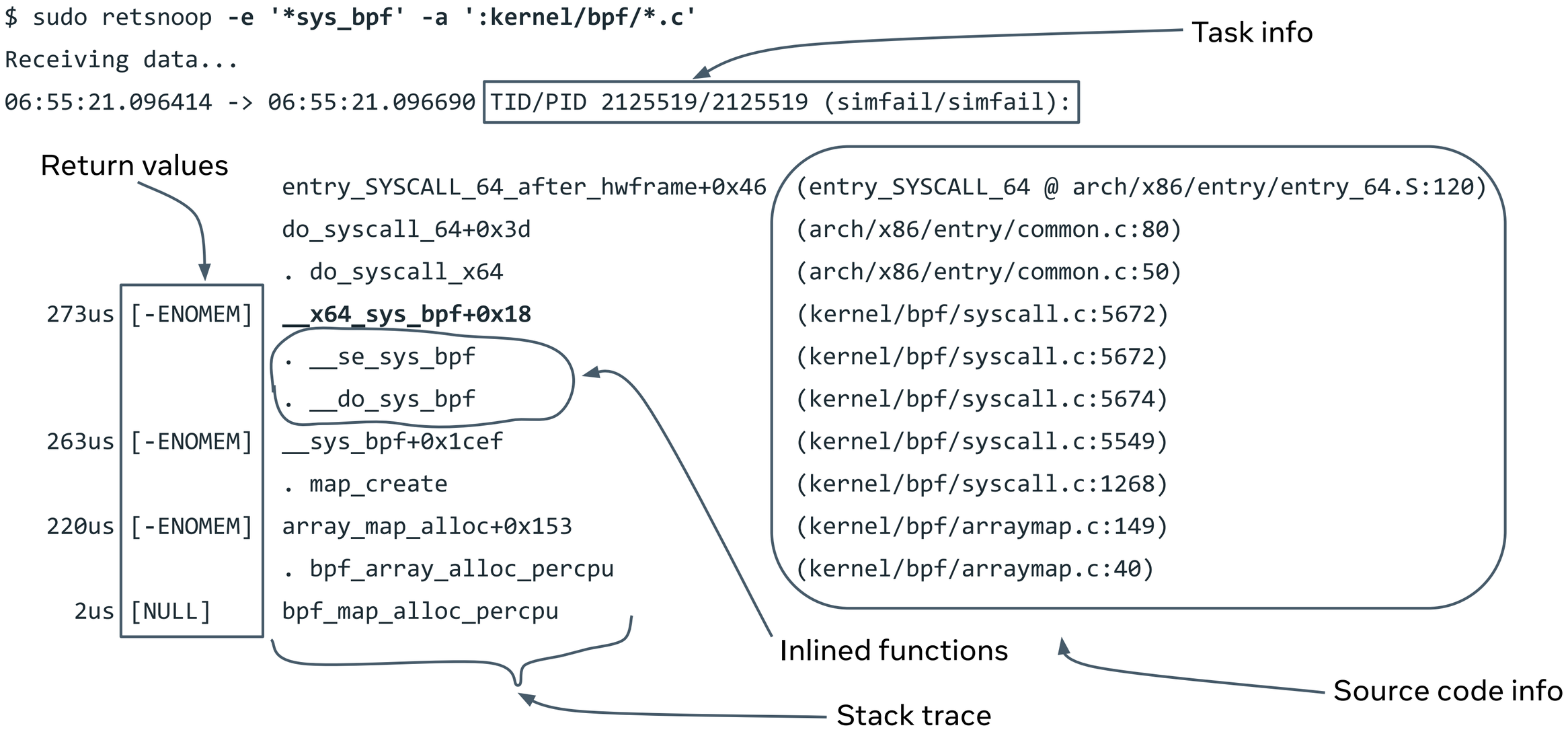
Task info

Return values

Source code info

Stack trace

Call stack mode (annotated)



Function filters

- Three sets of function specifiers:
  - Entry (**`-e, --entry`**)
  - Non-entry (**`-a, --allow`**)
  - Denied (ignored) (**`-d, --deny`**)
- Can be specified multiple times and are concatenated
- Denylist overrides entry and non-entry sets

Function filters

- Function name globs

  ```
  array_map_alloc, *sys_bpf, bpf_???_*
  ```

- Source code path globs

  ```
  :fs/proc.c, :kernel/bpf/*.c, :kernel/*/ioctl.c
  ```

- Kernel module globs

  ```
  * [nvidia*]
  ```

Tracing gotchas

- Intersected with **/sys/kernel/tracing/available_filter_functions**
- Beware of compiler smartness: **bpf_skb_is_valid_access.isra.63.part.64**
- **(!) Inlined functions are not traceable**
- Use verbose dry-run mode:

```
$ sudo retsnoop -v --dry-run -e ‘*sys_bpf’
...
Attached (dry run) to function #1 '__ia32_sys_bpf'.
Attached (dry run) to function #2 '__x64_sys_bpf'.
Attached (dry run) to function #3 'bpf_sys_bpf'.
Attached (dry run) to function #4 '__sys_bpf'
...
```

Context filters (errors)

- **(!) By default, retsnoop records only error stacks**
- Entry function has to return *error-like* result
  - **`NULL`** for pointer-returning functions
  - **`-Exxx`** for integer-returning functions
- Record all stacks with **`-S (--success-stacks)`**
- Tune error subset
  - **`-x (--allow-errors)`**
  - **`-X (--deny-errors)`**
  - E.g., `-x EINVAL -x E2BIG -x NULL`
  - Denylist takes precedence, of course

Context filters (process and latency)

- Filter **thread** by name
  - **`-n (--comm)`**
  - **`-N (--no-comm)`**
- Filter **process** by PID
  - **`-p (--pid)`**
  - **`-P (--no-pid)`**
- Filter by **entry function** duration
  - **`-L (--longer)`**
- All filters are **combinable**

LBR mode (--lbr)

```
$ sudo retsnoop -e '*sys_bpf' -a 'array_map_alloc_check' --lbr=any
Receiving data...
20:29:17.844718 -> 20:29:17.844749 TID/PID 2385333/2385333 (simfail/simfail):
...
[#22] ftrace_trampoline+0x14c                                          ->  array_map_alloc_check+0x5   (kernel/bpf/arraymap.c:53:20)
[#21] array_map_alloc_check+0x13  (kernel/bpf/arraymap.c:54:18)        ->  array_map_alloc_check+0x75  (kernel/bpf/arraymap.c:54:18)
      . bpf_map_attr_numa_node    (include/linux/bpf.h:1735:19)            . bpf_map_attr_numa_node    (include/linux/bpf.h:1735:19)
[#20] array_map_alloc_check+0x7a  (kernel/bpf/arraymap.c:54:18)        ->  array_map_alloc_check+0x18  (kernel/bpf/arraymap.c:57:5)
      . bpf_map_attr_numa_node    (include/linux/bpf.h:1735:19)
[#19] array_map_alloc_check+0x1d  (kernel/bpf/arraymap.c:57:5)         ->  array_map_alloc_check+0x6f  (kernel/bpf/arraymap.c:62:10)
[#18] array_map_alloc_check+0x74  (kernel/bpf/arraymap.c:79:1)         ->  __kretprobe_trampoline+0x0

                    entry_SYSCALL_64_after_hwframe+0x44  (arch/x86/entry/entry_64.S:112)
                    do_syscall_64+0x2d                   (arch/x86/entry/common.c:46)
   30us [-EINVAL]   __x64_sys_bpf+0x1c                   (kernel/bpf/syscall.c:4749)
   23us [-EINVAL]   __sys_bpf+0xca0                      (kernel/bpf/syscall.c:4629)
                    . map_create                         (kernel/bpf/syscall.c:863)
                    . find_and_alloc_map                 (kernel/bpf/syscall.c:123)
!    8us [-EINVAL]  array_map_alloc_check
```

LBR mode (--lbr) (annotated)

Important!

```
$ sudo retsnoop -e '*sys_bpf' -a 'array_map_alloc_check' --lbr=any
Receiving data...
20:29:17.844718 -> 20:29:17.844749 TID/PID 2385333/2385333 (simfail/simfail):
...
[#22] ftrace_trampoline+0x14c                                          ->  array_map_alloc_check+0x5   (kernel/bpf/arraymap.c:53:20)
[#21] array_map_alloc_check+0x13  (kernel/bpf/arraymap.c:54:18)        ->  array_map_alloc_check+0x75  (kernel/bpf/arraymap.c:54:18)
      . bpf_map_attr_numa_node    (include/linux/bpf.h:1735:19)            . bpf_map_attr_numa_node    (include/linux/bpf.h:1735:19)
[#20] array_map_alloc_check+0x7a  (kernel/bpf/arraymap.c:54:18)        ->  array_map_alloc_check+0x18  (kernel/bpf/arraymap.c:57:5)
      . bpf_map_attr_numa_node    (include/linux/bpf.h:1735:19)
[#19] array_map_alloc_check+0x1d  (kernel/bpf/arraymap.c:57:5)         ->  array_map_alloc_check+0x6f  (kernel/bpf/arraymap.c:62:10)
[#18] array_map_alloc_check+0x74  (kernel/bpf/arraymap.c:79:1)         ->  __kretprobe_trampoline+0x0
```

```
51 int array_map_alloc_check(union bpf_attr *attr)
52 {
53         bool percpu = attr->map_type == BPF_MAP_TYPE_PERCPU_ARRAY;
54         int numa_node = bpf_map_attr_numa_node(attr);
55
56         /* check sanity of attributes */
57         if (attr->max_entries == 0 || attr->key_size != 4 ||
58             attr->value_size == 0 ||
59             attr->map_flags & ~ARRAY_CREATE_FLAG_MASK ||
60             !bpf_map_flags_access_ok(attr->map_flags) ||
61             (percpu && numa_node != NUMA_NO_NODE))
62                 return -EINVAL;
```

Function call trace mode (-T)

```
$ sudo retsnoop -e 'bpf_prog_load' -a ':kernel/bpf/*.c' -T
Receiving data...

21:33:58.438961 -> 21:33:58.439338 TID/PID 8385/8385 (simfail/simfail):

FUNCTION CALL TRACE                                 RESULT                  DURATION
--------------------------------------------------  ----------------------  ---------
→ bpf_prog_load
    → bpf_prog_alloc
        ↔ bpf_prog_alloc_no_stats                   [0xffffc9000031e000]    5.539us
    ← bpf_prog_alloc                                [0xffffc9000031e000]   10.265us
    ↔ bpf_obj_name_cpy                              [11]                    2.169us
    → bpf_check
        [...]
    ← bpf_check                                     [0]                   293.563us
    → bpf_prog_select_runtime
        ↔ bpf_prog_alloc_jited_linfo                [0]                     2.943us
        → bpf_int_jit_compile
            [...]
        ← bpf_int_jit_compile                       [0xffffc9000031e000]   30.520us
        ↔ bpf_prog_jit_attempt_done                 [void]                  1.778us
    ← bpf_prog_select_runtime                       [0xffffc9000031e000]   42.725us
    → bpf_prog_kallsyms_add
        ↔ bpf_ksym_add                              [void]                  2.046us
    ← bpf_prog_kallsyms_add                         [void]                  6.104us
← bpf_prog_load                                     [5]                   374.697us
```

Function call trace mode (-T) (annotated)

```
$ sudo retsnoop -e 'bpf_prog_load' -a ':kernel/bpf/*.c' -T
Receiving data...

21:33:58.438961 -> 21:33:58.439338 TID/PID 8385/8385 (simfail/simfail):

FUNCTION CALL TRACE                                 RESULT                  DURATION
--------------------------------------------------  ----------------------  ---------
→ bpf_prog_load
    → bpf_prog_alloc
        ↔ bpf_prog_alloc_no_stats                   [0xffffc9000031e000]     5.539us
    ← bpf_prog_alloc                                [0xffffc9000031e000]    10.265us
    ↔ bpf_obj_name_cpy                              [11]                     2.169us
    → bpf_check
        [...]
    ← bpf_check                                     [0]                    293.563us
    → bpf_prog_select_runtime
        ↔ bpf_prog_alloc_jited_linfo                [0]                      2.943us
        → bpf_int_jit_compile
            [...]
        ← bpf_int_jit_compile                       [0xffffc9000031e000]    30.520us
        ↔ bpf_prog_jit_attempt_done                 [void]                   1.778us
    ← bpf_prog_select_runtime                       [0xffffc9000031e000]    42.725us
    → bpf_prog_kallsyms_add
        ↔ bpf_ksym_add                              [void]                   2.046us
    ← bpf_prog_kallsyms_add                         [void]                   6.104us
← bpf_prog_load                                     [5]                    374.697us
```

Function arguments capture

```
int cap_capable(const struct cred *cred,
                struct user_namespace *targ_ns,
                int cap,
                unsigned int opts);
```

Function arguments capture

```
$ sudo retsnoop -e '__sys_bpf' -a 'cap_capable' -T -x EPERM
07:58:29.853576 -> 07:58:29.853582 TID/PID 2369515/2369515 (veristat/veristat):
FUNCTION CALLS      RESULT    DURATION
-----------------   --------  --------
→ __sys_bpf
    ↔ cap_capable   [-EPERM]   0.301us
    ↔ cap_capable   [-EPERM]   0.215us
← __sys_bpf         [-EPERM]   3.862us
```

Function arguments capture

```
$ sudo retsnoop -e '__sys_bpf' -a 'cap_capable' -T -x EPERM -A -C args.fmt-mode=m
07:58:29.853576 -> 07:58:29.853582 TID/PID 2369515/2369515 (veristat/veristat):
FUNCTION CALLS      RESULT    DURATION
-----------------   --------  --------
→ __sys_bpf
                    › cmd = BPF_MAP_CREATE
                    › uattr = {{.kernel=0x7ffc521c62a0,.user=0x7ffc521c62a0}}
                    › size = 80
    ↔ cap_capable   [-EPERM]   0.885us
                    › cred = &{.usage={.counter=8},.uid={.val=1001},.gid={.val=984},.suid={.val=1001},…
                    › targ_ns = &{.flags=1,.keyring_name_list={…
                    › cap = 12
                    › opts = 0
    ↔ cap_capable   [-EPERM]   0.670us
                    › cred = &{.usage={.counter=8},.uid={.val=1001},.gid={.val=984},.suid={.val=1001},…
                    › targ_ns = &{.flags=1,.keyring_name_list={…
                    › cap = 21
                    › opts = 0
← __sys_bpf         [-EPERM]   5.620us
```



#define CAP_NET_ADMIN 12



#define CAP_SYS_ADMIN 21

Function arguments capture

- Captures **underlying struct data** for pointer args
- BTF-based pretty-printing
  - Argument names
  - Compact C-like struct/unions representations
    - *Skips zeroed fields*
  - Enums symbolized
  - Heuristics for integers to emit hex or dec formats
- Limits on data capture might truncate data
  - But can be adjusted, see `retsnoop --config-help` for **`args.*`** configs

Injected point probes

```
$ sudo retsnoop -e '*sys_execve' -a 'bprm_execve' -T -A -C args.fmt-mode=m \
     -J 'kprobe:bprm_execve+5' -J 'tp:sched:sched_process_exec'
08:54:40.041502 -> 08:54:40.041871 TID/PID 2530/2530 (zsh/zsh):
FUNCTION CALLS                                 RESULT   DURATION
-------------------------------------------    ------   ---------
→ __x64_sys_execve
                                               › regs = &{.r15=0x7ffc3021c470,.r13=0x55b7bfee78f0,.r12=0x7ffc3021c4f0,…
    → bprm_execve
                                               › bprm = &{.vma=0xffff88810d44c630,.vma_pages=1,.argmin=0x7fffffdff128,…
        ◎ kprobe:bprm_execve+0x5
                                               › regs = {.r12=0x55b7bfee78f0,.bp=0xffff888103d9fe00,.bx=0xffff888119021000,…
        ◎ tp:sched:sched_process_exec
                                               › filename = '/usr/bin/date'
                                               › pid = 2530
                                               › old_pid = 2530
    ← bprm_execve                               [0]     301.292us
← __x64_sys_execve                              [0]     359.827us
```

Injected point probes

- Supported types and captured context:
  - kprobes, optionally with offset, captures `struct pt_regs`
  - kretprobes, captures `struct pt_regs`
  - tracepoints, captures **formatted** tracepoint args
  - *raw* tracepoints, captures **unformatted** args

Injected point probes (verbose)

```
$ sudo retsnoop -e '*sys_execve' -a 'bprm_execve' -J 'kprobe:bprm_execve+5' -T -A -C args.fmt-mode=verbose
```

```
› regs = {
    .r12 = 0x55b7bfee78f0,
    .bp = 0xffff88810cf5dc00,
    .bx = 0xffff88811947d000,
    .r11 = 0xfefefefefefefeff,
    .r10 = 131064,
    .r9 = 0x8080808080808080,
    .r8 = 0x7f1ddb0cca68,
    .ax = 2,
    .si = 0x7f1ddb0cca6d,
    .di = 0xffff88810cf5dc00,
    .orig_ax = 0xffffffffffffffff,
    .ip = 0xffffffff8133b0c6,
    {
        .cs = 16,
        .csx = 0xffffc900009c0010,
        .fred_cs = {
            .cs = 16,
            .wfe = 1
        }
    },
    .flags = 514,
    .sp = 0xffffc900009c3ed0,
    {
        .ss = 24,
        .ssx = 24,
        .fred_ss = {
            .ss = 24
        }
    }
}
```

How to try retsnoop

- Packaged in **Fedora** and **Arch Linux**
- Prebuilt x86-64 and arm64 binaries in each Github release
  - https://github.com/anakryiko/retsnoop/releases

Source code and docs

- Source code and documentation:

  https://github.com/anakryiko/retsnoop

- “Tracing Linux kernel with retsnoop” blog post

  https://nakryiko.com/posts/retsnoop-intro/

- Check **`retsnoop --help`** and **`retsnoop --config-help`**